

Architecture Breakdown Document
MediAssist AI – Enterprise Healthcare
Agentic AI Copilot

1. Architecture Overview

MediAssist AI follows a modern enterprise-grade Agentic AI architecture designed to support healthcare information retrieval, document understanding, multimodal processing, and intelligent decision support. The architecture combines Retrieval-Augmented Generation (RAG), Multi-Agent Systems, Model Context Protocol (MCP), Vector Databases, and Large Language Models (LLMs) to provide a scalable and secure healthcare AI platform.

The system is organized into multiple logical layers, each responsible for a specific set of functions. These layers work together to process user requests, retrieve relevant information, execute enterprise tools, analyze medical documents, validate responses, and generate reliable outputs.

2. Architectural Design Principles

The architecture has been designed using the following principles:

- **Modular Design**

Each component performs a specific responsibility and can be developed, tested, and maintained independently.

- **Scalability**

The architecture supports increasing numbers of users, documents, databases, and AI agents without major redesign.

- **Security**

Database access is controlled through MCP tools rather than direct AI access.

- **Explainability**

Responses are generated using retrieved evidence and validated through evaluation mechanisms.

- **Observability**

Every stage of execution is monitored and tracked for debugging and performance analysis.

- **Cloud Readiness**

All services are designed to be containerized and deployed on Azure cloud infrastructure.

3. High-Level Architecture Layers

The architecture consists of seven primary layers:

1. User Interaction Layer
2. API Gateway Layer
3. Agent Orchestration Layer
4. Agent Processing Layer
5. Data & Knowledge Layer
6. AI Processing Layer
7. Monitoring & Observability Layer

Each layer contributes to the overall workflow and ensures smooth communication between components.

4. User Interaction Layer

Purpose

The User Interaction Layer acts as the entry point of the system.

Components

Streamlit Application

The Streamlit application provides an interactive interface for healthcare users.

Users can:

- Ask natural language questions
- Upload PDF documents
- Upload prescriptions
- Upload medical reports
- View AI-generated responses

Supported Users

Doctors

Access patient history, prescriptions, and lab reports.

Administrative Staff

Retrieve operational information and hospital workflows.

Insurance Teams

Verify billing records and insurance claims.

This layer focuses on user experience and interaction management.

5. API Gateway Layer

FastAPI Gateway

The FastAPI Gateway serves as the communication bridge between the frontend and backend AI services.

Responsibilities

- Receive user requests
- Validate incoming data
- Handle file uploads
- Manage API routing
- Forward requests to LangGraph orchestration
- Return processed responses

Benefits

- Separation of frontend and backend services
 - Better scalability
 - API standardization
 - Improved maintainability
-

6. Agent Orchestration Layer

LangGraph Supervisor

The LangGraph Supervisor functions as the central orchestration engine.

Purpose

It coordinates all AI agents and determines the execution sequence required to satisfy user requests.

Responsibilities

- Workflow management

- Agent coordination
- Task routing
- State management
- Result aggregation

The supervisor ensures that complex healthcare workflows can be executed efficiently through multiple specialized agents.

7. Agent Processing Layer

This layer contains the intelligent agents responsible for specific tasks.

7.1 Planner Agent

Purpose

The Planner Agent analyzes user intent and decomposes complex requests into manageable subtasks.

Responsibilities

- Query understanding
- Intent detection
- Task decomposition
- Workflow planning
- Agent selection

Example

User Request:

"Analyze this medical report and retrieve patient history."

Planner Agent creates tasks:

1. Analyze uploaded report
 2. Retrieve patient history
 3. Generate summary
 4. Validate output
-

7.2 Retriever Agent

Purpose

The Retriever Agent retrieves contextual information from enterprise documents.

Responsibilities

- Semantic search
- Vector similarity search
- Document ranking
- Context retrieval

Data Sources

- Hospital SOPs
- Compliance documents
- Insurance policies
- Clinical guidelines
- Operational manuals

The retrieved information serves as grounding context for response generation.

7.3 MCP Agent

Purpose

The MCP Agent enables secure interaction with enterprise healthcare systems.

Responsibilities

- Tool execution
- Database access
- Query processing
- Structured data retrieval

Example MCP Tools

- `get_patient_history()`
- `get_appointments()`
- `get_lab_results()`

- `get_prescriptions()`
- `get_billing_summary()`

This architecture prevents direct database exposure to AI models.

7.4 Vision Agent

Purpose

The Vision Agent processes medical images and visual healthcare documents.

Responsibilities

- OCR processing
- Image understanding
- Information extraction
- Medical report interpretation

Input Types

- Prescriptions
- Medical reports
- Lab reports
- Scanned healthcare documents

The extracted information is converted into structured content for downstream processing.

7.5 Reasoning Agent

Purpose

The Reasoning Agent synthesizes information from multiple sources.

Inputs

- User queries
- Retrieved documents
- Database records
- Image analysis results

Responsibilities

- Context fusion

- Logical reasoning
- Insight generation
- Recommendation generation

This agent acts as the intelligence layer of the system.

7.6 Evaluator Agent

Purpose

The Evaluator Agent validates the quality and reliability of generated outputs.

Responsibilities

- Hallucination detection
- Grounding verification
- Relevance validation
- Safety assessment
- Confidence scoring

Benefits

Ensures that generated healthcare information remains trustworthy and evidence-based.

7.7 Response Agent

Purpose

The Response Agent prepares the final output delivered to users.

Responsibilities

- Output formatting
- Response polishing
- Guardrail enforcement
- Content moderation
- Final delivery

This agent acts as the final checkpoint before information reaches the user.

8. Data & Knowledge Layer

Vector Database

Technologies

- FAISS
- ChromaDB

Purpose

Stores document embeddings for semantic retrieval.

Functions

- Similarity search
 - Context retrieval
 - Knowledge grounding
-

PostgreSQL Database

Purpose

Stores structured healthcare data.

Major Tables

- Patients
- Appointments
- Admissions
- Billing
- Lab Results
- Prescriptions
- Departments

The database serves as the primary enterprise data repository.

MCP Server

The MCP Server acts as a secure integration layer between AI agents and enterprise databases.

Benefits include:

- Secure tool execution

- Controlled database access
 - Standardized integrations
 - Enterprise-grade architecture
-

9. AI Processing Layer

Large Language Models

OpenAI Models

Used for reasoning, summarization, planning, and response generation.

Groq Models

Used for low-latency inference and high-performance AI processing.

Embedding Models

Generate vector representations of healthcare documents for semantic retrieval.

10. Ingestion Pipeline

The ingestion pipeline prepares enterprise documents for retrieval.

Steps

Document Upload

Healthcare documents are uploaded into the system.

Chunking

Large documents are divided into smaller segments.

Embedding Generation

Chunks are converted into vector representations.

Indexing

Vectors are stored within FAISS or ChromaDB.

This pipeline forms the foundation of the Enterprise RAG system.

11. Observability Layer

LangSmith

LangSmith provides monitoring and observability capabilities.

Features

- Agent tracing
- Workflow monitoring
- Execution debugging
- Performance tracking
- Evaluation analysis

This layer helps developers understand and optimize system behavior.

12. End-to-End Request Flow

Step 1:

User submits query through Streamlit.

Step 2:

FastAPI receives request.

Step 3:

LangGraph Supervisor analyzes workflow.

Step 4:

Planner Agent creates subtasks.

Step 5:

Retriever Agent retrieves enterprise documents.

Step 6:

MCP Agent retrieves database records.

Step 7:

Vision Agent analyzes uploaded images.

Step 8:

Reasoning Agent synthesizes information.

Step 9:

Evaluator Agent validates output.

Step 10:

Response Agent formats response.

Step 11:
Final response is returned to the user.

13. Architectural Advantages

- Enterprise-grade design
 - Secure database access
 - Modular agent architecture
 - Multimodal processing support
 - Scalable deployment model
 - Explainable AI workflow
 - Strong observability capabilities
 - Cloud-native architecture
 - Production-ready deployment strategy
-

14. Conclusion

The MediAssist AI architecture represents a modern enterprise AI platform that integrates RAG, Multi-Agent Systems, MCP-based tool orchestration, Multimodal AI, and cloud-native deployment practices. The architecture provides a scalable, secure, and intelligent foundation for healthcare information retrieval, workflow automation, and decision support while maintaining high standards of reliability, explainability, and operational efficiency.